JSONata

https://docs.jsonata.org/overview.html

1 Herramientas de transformación para JSON 2
2 Introducción a JSONata 2
3 Fundamentos de JSONata 3
3.1 Sintaxis básica 3
3.2 Expresiones básicas y operadores 5
4 Selección de Datos 5
5 Transformación de Datos 6
5.1 Creación de nuevos objetos 6
5.2 Mapeo de estructuras JSON 7
6 Funciones en JSONata 8
6.1 Funciones de librería 8
6.2 Uso de variables 12
6.3 Definir funciones personalizadas 13
6.4 Expresiones lambda (funciones anónimas) 14
7 Expresiones Avanzadas 14
7.1 Uso de expresiones condicionales ($booleano ? valor1 : valor2) 15
7.2 Expresiones regulares ($match, $replace y operador ~=) 16
8 Ejemplos Prácticos 18
8.1 Consultas de datos: 19
8.2 Transformación de documentos 22

1 Herramientas de transformación para JSON

A la hora de transformar contenido en JSON tenemos varias alternativas, que trabajan a diferente nivel de abstracción:

- **ETL (Extract, Transform, Load)**. Trabajan a un nivel de **abstracción alto**. Son herramientas que están diseñadas para la ingesta, enrutado transformación y monitorización de datos a cualquier escala. Son capaces de trabajar con datos en cualquier formato (JSON, XML, CSV, bases de datos relacionales, bases de datos NoSQL, datos en tiempo real,….) y llevarlos a cualquier otro formato. Algunas de estas herramientas son:
  - **Open Source:** Apache NiFi, Apache Hop.
  - **Streaming en tiempo real:** Apache Kafka, Flink.
  - **Cloud:** AWS Glue, Google Dataflow, Azure Data Factory

- **Frameworks y herramientas declarativas específicas para JSON.** Trabajan a un nivel de **abstracción medio**. Se trata de herramientas y tecnologías mucho más sencillas que las anteriores, que se centran únicamente en procesar contenido JSON. Existen varias alternativas, siendo las más populares:
  - **JSONata:** Es la que vamos a ver en este tutorial.
  - **JOLT:** Ofrece una funcionalidad parecida a ***JSONata***, pero con una sintaxis un poco más compleja y alguna limitación más.
  - **JQ:** Es una herramienta de línea de comandos muy flexible para filtrar, modificar y reestructurar JSON. Ideal para procesos automatizados y scripts.

- **APIs específicas de lenguajes de programación.** Trabajan a un nivel de **abstracción bajo**. Los principales lenguajes de programación como Java, Python, C#, etc, incorporan librerías específicas para trabajar con ficheros JSON. Esta forma de trabajar nos proporcionará un control total sobre su procesamiento, pero requiere de mayor trabajo a nivel técnico.

2 Introducción a JSONata

***JSONata (JSON Query and Transformation Language)* es un lenguaje de consulta y transformación de datos diseñado específicamente para JSON**. Su propósito principal es permitir la manipulación eficiente de estructuras JSON sin necesidad de escribir código complejo en otros lenguajes de programación.

Sus características clave son:

- Tiene una sintaxis expresiva y declarativa.
- Capacidad para filtrar, transformar y mapear datos JSON.
- Compatible con funciones matemáticas, cadenas, arrays y estructuras anidadas.
- Soporte para funciones personalizadas y expresiones condicionales.

JSONata puede usarse en múltiples entornos, como **Node.js, bases de datos NoSQL** y **herramientas de integración de datos.** También, existen plataformas online para interactuar con esta tecnología como:

https://try.jsonata.org/

JSONata se parece a JSONPath en el sentido de que ambos permiten la consulta de datos JSON, pero JSONata **ofrece más funcionalidades**, como transformaciones avanzadas y expresiones personalizadas. **JSONata no solo permite consultar JSON, sino también transformarlo**, lo que lo hace más potente que JSONPath para tareas de procesamiento de datos.

Característica	JSONata	JSONPath
Consulta de datos JSON	✅ Sí	✅ Sí
Filtrado avanzado	✅ Sí	⚠️ Limitado
Transformaciones de datos	✅ Sí	❌ No
Funciones personalizadas	✅ Sí	❌ No
Expresiones condicionales	✅ Sí	❌ No

Los casos de uso más comunes para esta tecnología son:

- Filtrado de datos.
- Transformación de estructuras JSON en otros formatos.
- Generación de informes dinámicos.
- Integración con sistemas NoSQL.
- Automatización en herramientas de integración de datos.

3 Fundamentos de JSONata

3.1 Sintaxis básica

JSONata usa una sintaxis declarativa. Algunas características esenciales de su sintaxis son:

- **Acceso directo a propiedades** (basta con poner su nombre).
- **Uso de expresiones para filtrar y transformar datos**.
- **Funciones y operadores** para manipular valores.
- **Expresiones anidadas** para realizar consultas avanzadas.

La expresión más básica es la que te permite recuperar el valor de una propiedad. Para ello, bastará con escribir simplemente su nombre al estilo que usábamos en JSONPath. Por ejemplo:

Si contamos con el siguiente JSON:

```
{
 "nombre": "Carlos",
 "edad": 40,
 "contacto": {
  "email": "carlos@example.com",
  "telefono": "555-1234"
 },
 "hobbies": ["fútbol", "cine", "cocina"],
 "activo": true,
 "proyectos": [
  {
   "titulo": "Desarrollo de Aplicación",
   "completado": false,
   "lenguajes": ["Python","Java","C#"]
  },
  {
   "titulo": "Diseño de Página Web",
   "completado": true,
   "lenguajes": ["Python","Java"]
  }
 ]
}
```

Podríamos escribir las expresiones:

- Edad
  Resultado : *40*

- contacto.email
  Resultado : *carlos@example.com*

- proyectos[1].lenguajes
  Resultado : [“Python”, “Java”]

- contacto.*
  Resultado: *["carlos@example.com", "555-1234"]*

Los tipos de datos con los que trabaja JSONata son los propios de los documentos JSON:

- Números.
- Strings.
- Booleanos.
- Objetos.
- Arrays.
- Valores nulos.

3.2 Expresiones básicas y operadores

Las expresiones de JSONata pueden incluir diferentes tipos de operadores:

- **Aritméticos**: + - * / % (módulo)
- **Comparación:** = != > < >= <= **in[v1,v2,…]** ~= (para expresiones regulares)

  OJO, a diferencia de JSONPath, la comparación igual se representa con un único =.
- **Lógicos**: and or not
- **Concatenación de cadenas**. &
- **Agrupación de expresiones**. ( )

4 Selección de Datos

Para acceder a una propiedad de un objeto en JSONata, se usa la notación de punto ., igual a JSONPath:

```
contacto.email
```

Si lo que queremos es **filtrar los elementos de un array**, debemos poner entre [ ] la condición que se debe cumplir. Por ejemplo:

```
proyectos[completado=true]
```

Dentro de una condición, podemos utilizar expresiones compuestas si usamos *operadores lógicos* para conectarlas. Por ejemplo:

```
proyectos[completado=true and $count(lenguajes)>1]
```

Devolvería:

```
{
  "titulo": "Diseño de Página Web",
  "completado": true,
  "lenguajes": [ "Python", "Java" ]
}
```

El comodín * , al igual que ocurría en JSONPath, nos permite seleccionar múltiples elementos dentro de un objeto sin necesidad de conocer los nombres de las claves. Así, la expresión

```
contacto.*
```

Devolvería: ["carlos@example.com", "555-1234"]

Para acceder a las distintas posiciones de un array, utilizamos un índice numérico entre corchetes [índice]. El primer valor se corresponde con el índice 0 y se pueden utilizar valores negativos para desplazarse desde el final al principio.

```
hobbies[0]

proyectos[-1].lenguajes[0]
```

JSONata no permite, de forma nativa, usar una sintaxis de rango como [1:3] tal como lo hace JSONPath.

5 Transformación de Datos

JSONata no sólo permite consultar datos, sino que también ofrece potentes herramientas para transformar estructuras JSON. En este apartado exploraremos tres aspectos fundamentales:

- **Creación de nuevos objetos**
- **Mapeo de estructuras JSON**

5.1 Creación de nuevos objetos

En JSONata, **para crear un nuevo objeto se utilizan llaves { }**, de manera similar a cómo se define un objeto en JSON. **Dentro de las llaves definiremos pares clave-valor**, donde cada valor puede ser una expresión dinámica basada en los datos de entrada, un cálculo o el resultado de una función.

Por ejemplo, si partimos del siguiente JSON:

```
{
  "usuario": {
    "nombre": "Carlos",
    "apellido": "Pérez",
    "edad": 30
  }
}
```

Podríamos crear un nuevo objeto de la siguiente forma:

```
{
  "nombreCompleto": $uppercase(usuario.nombre) & " " & $uppercase(usuario.apellido),
  "esMayorDeEdad": usuario.edad >= 18
}
```

Y obtendríamos el siguiente resultado.

```
{
  "nombreCompleto": "CARLOS PÉREZ",
  "esMayorDeEdad": true
}
```

5.2 Mapeo de estructuras JSON

El mapeo consiste en recorrer y transformar elementos de un array de objetos JSON, para generar una nueva estructura con la forma que necesitemos.

Supongamos que tenemos el siguiente JSON con una lista de productos:

```
{
  "productos": [
    { "id": 1, "nombre": "Laptop", "precio": 1200 },
    { "id": 2, "nombre": "Mouse", "precio": 25 },
    { "id": 3, "nombre": "Teclado", "precio": 45 }
  ]
}
```

Si deseamos obtener un nuevo array con solo los nombres de los productos, podríamos utilizar:

```
productos.nombre
```

Que devolvería:

```
[
  “Laptop”,
  “Mouse”,
  “Teclado”
]
```

Si lo que quisiéramos es transformar cada objeto para incluir, por ejemplo, el precio con IVA (digamos un 21%) y renombrar algunas propiedades, podríamos hacer lo siguiente:

```
productos.{
    "ProductoID": id,
    "Descripción": nombre,
    "PrecioConIVA": precio * 1.21
}
```

Que nos devolvería:

```
[
  { "ProductoID": 1, "Descripción": "Laptop", "PrecioConIVA": 1452 },
  { "ProductoID": 2, "Descripción": "Mouse", "PrecioConIVA": 30.25 },
  { "ProductoID": 3, "Descripción": "Teclado", "PrecioConIVA": 54.45 }
]
```

La notación `productos.{ ... }` indica que para cada elemento del array se creará un nuevo objeto. Dentro de las llaves se pueden definir propiedades, asignándoles valores o expresiones basadas en los datos originales.

6 Funciones en JSONata

6.1 Funciones de librería

JSONata incluye un amplio conjunto de funciones nativas que facilitan la manipulación de datos (funciones matemáticas, de texto, de arreglos, etc.). El uso de estas funciones resulta clave para lograr dotar de versatilidad y operatividad a esta tecnología. A continuación, se muestran algunas de ellas organizadas por categoría:

- **Funciones matemáticas**.
    - **$sum(array)**. Suma todos los elementos numéricos de un array.

      $sum([10, 20, 30])

    - **$avg(array)**. Calcula el promedio de los valores en un array.

      $avg[10, 20, 30])

    - **$min(array) y $max(array)**. Devuelven el valor mínimo o máximo del array.

      $min[10, 20, 30])

    - **$round(valor [, numDecimales])**. Redondea un número al entero más cercano o utilizando un número de decimales determinado.

      $round(3.14159, 2)

- **Funciones de texto**.
    - **$uppercase(texto) y $lowercase(texto)**. Convierte a mayúsculas o minúsculas un texto.

      $uppercase("hola")

    - **$length(texto)**. Devuelve la longitud de un texto.

      $length("hola")

    - **$substring(texto, inicio, [longitud])**. Extrae una subcadena de un string desde una posición determinada hasta el final, o indicándole el número de caracteres.

      $substring("JSONata",0,4)

    - **$trim(texto)**. Elimina espacios en blanco al inicio y al final de la cadena.

      $trim(" Hola a todos ")

    - **$contains(texto, patrón)**. Devuelve true si el patrón forma parte del texto. El patrón puede pasarse en forma de String o mediante una *expresión regular*.

      $contains("Hola mundo", /MUNDO/i)
      (OJO) /i es para que no sea sensible a mayúsculas y minúsculas.

- **$split(texto, separador [,límite])**. Devuelve un array con las diferentes partes que componen una cadena, considerando como separador el indicado. Si el texto no es pasado, el valor del contexto será el pasado en su lugar. Por ejemplo:

```
$split("1:fernando:garcía castillo:12345678H",":")
```

```
[
  "1",
  "fernando",
  "garcía castillo",
  "12345678H"
]
```

- **$join(array [, separador])**. Hace lo opuesto a *$split*. en este caso, se le pasa un array de Strings con los valores y lo que hace es devolver un String en el que aparezcan todos utilizando el *separador* entre ellos. Si no se le pasa el parámetro separador, se pondrán pegados uno tras otro.

```
$join(["1","2","3","4"],":")
```

```
"1:2:3:4"
```

- **$replace(texto, patrón, reemplazo)**. Reemplaza partes de la cadena que coincidan con un patrón (que puede ser una expresión regular).

```
$replace("a-b-c", "-", "_")
```

- **Funciones numéricas.**

  - **$number(arg)**. Castea el argumento pasado a un número. Si no le pasa el argunmento, se usará el valor del contexto en que estemos.

```
$number("5")
["1","2","3"].$number()
```

  - **$abs(number)**. Devuelve el valor absoluto.
  - **$floor(number)**. Devuelve la parte entera de un valor decimal.
  - **$ceil(number)**. Devuelve el entero que esté por encima del valor decimal pasado.
  - **$round(number[, precision]).** Redondea el número pasado.
  - **$power(base, exponente).** Calcula la potencia.
  - **$sqrt(number).** Calcula la raíz cuadrada.
  - **$random().** Devuelve un número aleatorio entre [0 y 1)

- **Funciones de arrays**.

  - **$count(array)**. Devuelve la longitud del array.

```
$count([1,2,3])
```

- **$append(array1, array2)**. Retorna un array en el que al final de array1 se le añade array2. Si cualquiera de los parámetros no es un array, se convertirá a un array de un único elemento.

```
$append([1,2,3], [4,5,6])
$append([1,2,3], 4)
$append([1,2,3], "hola")
```

- **$sort(array [, function])**. Retorna un array que contiene todos los valores del array pasado ordenados. Si no se le proporciona ninguna función, el array pasado debe ser únicamente de números o strings. Si le pasa una función, debe tener el siguiente prototipo:

```
function(left,right)
```

  Esta función interviene en el proceso de ordenación y será invocada continuamente por la función $sort, pasándole pares de elementos en sus dos parámetros. Si el que está en *left*, se considera mayor que el que está en *right*, entonces debe devolver *true* y *false* en caso contrario.

```
$sort([3,1,2])
$sort(["hola","buenos","días"])
$sort(Factura.Productos, function($l, $r) {
                                $l.peso > $r.peso   })
```

- **$reverse(array)**. Invierte la posición de los elementos del array.

```
$reverse([1,2,3])
```

- **$shuffle(array)**. Cambia el orden de forma aleatoria de sus elementos.

```
$shuffle([1..10])
```

- **$distinct(array)**. Quita los elementos repetidos del array.

```
$distinct([1,3,2,1,2,4])
```

- **$map(array, function).** Retorna un array que contiene el resultado de aplicar una función pasada por parámetro a cada valor del array pasado en el primer parámetro.

  La función pasada en el segundo parámetro puede tomar uno de los siguientes formatos:

  - Si solo recibe un parámetro, coincidirá con el valor procesado en ese momento.

```
$map([1..5], $string) => ["1", "2", "3", "4", "5"]
```

    En el ejemplo anterior, estamos utilizando la función predefinida **$string** que recibe por parámetro el valor que sea y retorna el valor convertido a String.

- Si recibe dos parámetros, el primero será el valor a procesar y el segundo será el índice de ese valor dentro del array.

- Si recibe tres parámetros, el primero será el valor a procesar, el segundo será el índice de ese valor dentro del array y el tercero será el array entero.

```
$map(Email.address, function($v, $i, $a) {
  'Item ' & ($i+1) & ' of ' & $count($a) & ': ' & $v
})
```

- **$filter(array, function).** Retorna un array con los elementos del array pasado por parámetro que satisfacen la función pasada en el segundo parámetro. La función puede adoptar cualquiera de los formatos vistos en la función **$map**, y debe retornar **true**, cuando el valor supera el filtro.

```
$filter(Account.Order.Product, function($v, $i, $a) {
 $v.Price > $average($a.Price)
})
```

- **$reduce(array, function [,init]).** Se utiliza para iterar sobre un array y acumular los resultados que vamos obteniendo en un único valor. La función recibirá al menos dos parámetros:

```
function(accumulator, value [, index [, array]] )
```

¡OJO! Si el parámetro opcional *init* no es proporcionado, el valor inicial que recibe el parámetro *accumulator* coincidirá con el del primer elemento del array y la iteración comenzará desde el segundo elemento, en otro caso, el parámetro *accumulator* tomará inicialmente el valor de *init* y la iteración comenzará por el primer elemento.

```
$reduce([1..5], function($i, $j){$i * $j})      // Retorna 120
$reduce([1..5], function($i, $j){$i * $j})      // Retorna 12000
```

- **Funciones de objetos**.

  - **$keys(object)**. Devuelve un array que contiene las claves del objeto pasado.

```
$keys(contacto).{}
```

  - **$lookup(object, key)**. Devuelve el valor asociado con la clave pasada en el objeto. Si el primer argumento es un array, devolverá se devolverán todos los valores que coincidan con esa clave.

```
$lookup(proyectos, "titulo")
```

```
[
  "Desarrollo de Aplicación",
  "Diseño de Página Web"
]
```

- **$each(object, function)**. Retorna un array que contiene todos los valores retornados por la función que se aplicará a cada par clave/valor del objeto. una función, debe tener el siguiente prototipo:

  function(value,name)

```
$each(contacto, function($valor,$clave){
        $clave & "__" & $valor}
)
```

```
[
  "email__carlos@example.com",
  "telefono__555-1234"
]
```

- **Funciones de fecha y hora**.
  - **$now()**. Retorna la fecha y hora actual en formato ISO 8601.
  - **$millis()**. Retorna el timestamp con la fecha y hora actual en milisegundos.
  - **$fromMillis(millis, format)**. Dado que no hay un tipo de datos específico para fechas y horas en JSON, es muy probable que el formato de las funciones anteriores no sean el que nos interese. Esta función permite expresar un timestamp en el formato que le indiquemos en el segundo parámetro:

    Fecha actual:
    $fromMillis($millis(),'[M01]/[D01]/[Y]')

    Hora actual:
    $fromMillis($millis(),'[H01]:[m01]:[s01]') // Hora actual

  - **$toMillis(dateTimeString [, picture])**. Convierte a milisegundos el string con la fecha y hora pasado. Podemos especificar en el formato *picture*, el formato en el que está.

6.2 Uso de variables

JSONata permite el uso de variables, y se pueden definir utilizando el operador := . **Las variables en JSONata no necesitan ser declaradas previamente y pueden cambiar de tipo en tiempo de ejecución**. Una vez definidas, podemos usarlas su contenido como parte de nuestras expresiones.

La sintaxis para definir una variable es la siguiente:

$variable := expresión

Donde *expresión* puede tomar cualquier valor de los tipos definidos en JSON, así como, puede referenciar también a una función determinada. Por ejemplo:

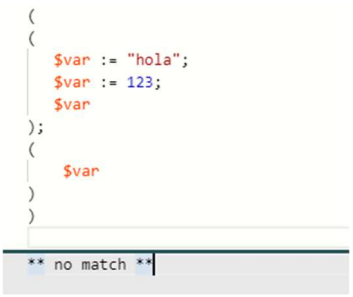
```
$varX := "hola"
$varX := 123
$varX := [1,2,3]
$varX := {"c1": "v1", "c2": "v2"}
$varX := null

(
 $f1 := function($i, $j){$i * $j};
 $reduce([1..5], $f1 ,100)
)
```

Las variables también se pueden definir dentro de funciones para almacenar valores temporales.

```
(
$sumar := function($a, $b) {
      ($resultado := $a + $b;
       $resultado)
};
$sumar(3,4)
)
```

Las variables en JSONata tienen **ámbito local** dentro de una expresión o función y no pueden ser usadas fuera de donde fueron declaradas.



6.3 Definir funciones personalizadas

En JSONata podemos definir funciones personalizadas para encapsular lógica compleja, reutilizar código y mejorar la claridad de nuestras expresiones. Esto se logra mediante la asignación de la función a una variable usando el operador :=.

La forma general de definir una función es:

```
$nombreFunc := function($param1, $param2, ...) {
 // Cuerpo de la función: expresión(s) que se evalúan
 expresión_resultado
}
```

Nota: **Si la función tiene más de una expresión, se deben separar con punto y coma (;) y el valor de la última expresión será el resultado devuelto.**

Las variables definidas dentro de la función tienen ámbito local y no afectan al entorno global.

Por ejemplo:

```
(
  $calcularAreaConDescuento := function($ancho, $alto, $descuento) {
    (
     $area := $ancho * $alto;
     $area - ($area * $descuento)
    )
  };

  $calcularAreaConDescuento(10, 5, 0.1)
)
```

6.4 Expresiones lambda (funciones anónimas)

Las ***expresiones lambda*** (o *funciones anónimas*) en JSONata son funciones definidas "sobre la marcha", sin asignarlas necesariamente a una variable. Estas expresiones permiten escribir funciones *inline*, lo que es muy útil para transformar o filtrar colecciones, sin tener que definir una función global completa.

La forma general de definir una *función anónima* en JSONata es:

```
function($param1, $param2, …) {
 // Cuerpo de la función: una o varias expresiones
 expresión
}
```

Si la función consta de una sola expresión, el resultado de esa expresión se convierte en el valor de retorno de la función. **Si el cuerpo es más complejo (varias operaciones o declaraciones), se pueden encerrar entre paréntesis () y separarlas con ;** .

El uso más frecuente es aplicarlas a otras funciones como *$map*, *$filter* o *$reduce*. Por ejemplo:

```
$map([1, 2, 3, 4], function($x) { $x * 2 })

$filter([10, 15, 20, 25], function($x) { $x > 18 })

$reduce([1, 2, 3, 4, 5], function($acc, $x) { $acc + $x }, 0)
```

7 Expresiones Avanzadas

En este apartado exploraremos algunos tipos de expresiones avanzadas en JSONata, que nos permitirán realizar transformaciones de datos más dinámicas y flexibles.

7.1 Uso de expresiones condicionales ($booleano ? valor1 : valor2)

JSONata, como hacen JavaScript y otros lenguajes, proporciona una sintaxis condicional basada en siguiente operador **ternario**.

```
condición ? valor_si_true : valor_si_false
```

Por ejemplo:

```
(
  $edad := 20;
  $edad >= 18 ? "Mayor de edad" : "Menor de edad"
)
```

También, podemos integrar la condición en expresiones más complejas:

```
(
  $usuario := { "nombre": "Ana", "edad": 16 };
  {
    "nombre": $usuario.nombre,
    "tipo": $usuario.edad >= 18 ? "Adulto" : "Menor"
  }
)
```

Existe la posibilidad de anidar múltiples condiciones, de forma, que seamos capaces de representar los típicos ***if-anidados*** de otros lenguajes.

```
(
$temperatura := 30;
$temperatura > 35 ? "Calor extremo" :
$temperatura > 25 ? "Caluroso" :
$temperatura > 15 ? "Templado" :
"Frío"
)
```

Cuando la condición de la expresión se evalúa a *null*, se considerará que no es true, por lo que entrará por la segunda parte.

```
(
  $usuario := { "nombre": "Carlos" };
  $usuario.edad ? $usuario.edad : "Edad no especificada"
)
```

7.2 Expresiones regulares ($match, $replace y operador ~=)

Las expresiones regulares en JSONata permiten realizar **búsquedas, reemplazos y validaciones** dentro de cadenas de texto.

La función **"match"**, aplica un patrón, expresado mediante una expresión regular, a un string y devuelve un array de objetos con todas las coincidencias que haya encontrado. Su sintaxis es la siguiente:

```
$match(str, pattern [, limit])
```

Donde *limit* hace referencia al número máximo de objetos que devolverá.

Si el parámetro str no es proporcionado, el valor que tenga el contexto en ese momento será el que se tenga en cuenta.

Los objetos que devuelve la función tienen los siguientes campos:

- **match** – El substring encontrado que concuerda.
- **index** - El offset (comenzando en 0) en el que comienza el match.
- **groups** – Si la expresión regular contiene grupos (paréntesis), este campo contiene un array de los strings que lo conforman.

Por ejemplo, imaginemos que queremos sacar los distintos números que aparecen en un texto. Podríamos usar la siguiente expresión:

```
(
    $texto := "Pondremos como primer número el 11. También, pondremos el 22 y el 33";
    $match($texto, /\d+/)
)
```

```
[
  {
    "match": "11",
    "index": 32,
    "groups": []
  },
  {
    "match": "22",
    "index": 58,
    "groups": []
  },
  {
    "match": "33",
    "index": 66,
    "groups": []
  }
]
```

Si, por ejemplo, quisiéramos recuperar en un String todos los números encontrados, podemos usar la siguiente expresión:

```
(
    $texto := "Pondremos como primer número el 11. También, pondremos el 22 y el 33";
    $join( $match($texto, /\d+/).match, ",")
)
```

```
"11,22,33"
```

Es posible también, usar la función **"$replace"** para hacer sustituciones de ciertas partes de un String por un valor determinado. Esta función devuelve un String y recibe como parámetros, el texto que queremos utilizar como base, la *expresión regular* que identifique a lo que queremos sustituir y el valor por el que queremos hacer la sustitución. La sintaxis de la función es:

```
$replace(str, pattern, replacement [, limit])
```

Por ejemplo, imaginemos que queremos sustituir en el texto anterior los números por una X

```
(
    $texto := "Pondremos como primer número el 11. También, pondremos el 22 y el 33";
    $replace($texto, /\d+/,"X")
)
```

```
"Pondremos como primer número el X. También, pondremos el X y el X"
```

Por último, vamos a ver cómo podemos usar el operador `~>` para verificar si un texto cumple una determinada *expresión regular*. Por ejemplo:

```
(
    $dni := "123456789-H";
    $dni ~> /^[0-9]?[0-9]{7}-?[A-Z]$/ ? "DNI válido" : "Formato incorrecto"
)
```

```
"Formato incorrecto"
```

8 Ejemplos Prácticos

Dado el fichero de ejemplo ***datosPaises.json***, cuya estructura es la siguiente:


```
{
 "paises": [
  {
   "nombre": "España",
   "superficie": 505990,
   "continente": "Europa",
   "numero_de_habitantes": 46754783,
   "capital": "Madrid",
   "principales_ciudades": [
    {
     "nombre": "Madrid",
     "poblacion": 3277024
    },
    {
     "nombre": "Barcelona",
     "poblacion": 1620343
    },
    {
     "nombre": "Valencia",
     "poblacion": 794288
    },
    {
     "nombre": "Sevilla",
     "poblacion": 688711
    }
   ],
   "idiomas": [
    {
     "idioma": "Español",
     "porcentaje": 99
    },
    {
     "idioma": "Catalán",
     "porcentaje": 17
    },
    {
     "idioma": "Gallego",
     "porcentaje": 5
    },
    {
     "idioma": "Euskera",
     "porcentaje": 3
    }
   ]
  },
  ...
  ... ]
}
```

8.1 Consultas de datos:

1) Obtener los nombres de los países de “Europa”

```
paises[continente="Europa"].nombre
```

```
[
  "España",
  "Francia"
]
```

2) Obtener los nombres y población de los países europeos con el formato Nombre(población)

```
$map(paises, function($v) {
   $v.continente="Europa" ? $v.nombre & "(" & $v.numero_de_habitantes & ")"
})
```

```
[
  "España(46754783)",
  "Francia(67081000)"
]
```

También con:

```
paises[continente="Europa"].(nombre & " (" & numero_de_habitantes & ")")
```

3) Los nombres de los países en los que se habla inglés

```
paises[idiomas[idioma="Inglés"]].nombre
```

```
[
  "Estados Unidos",
  "India",
  "Australia"
]
```

4) Obtener los nombres y población de las ciudades europeas con más de 1000000 de habitantes con el formato { “ciudad”: “x” , “habitantes”: y}

```
paises[continente="Europa"].principales_ciudades[poblacion>1000000].{"ciudad":nombre, "habitantes":poblacion}
```

```
[
  {
    "ciudad": "Madrid",
    "habitantes": 3277024
  },
  {
    "ciudad": "Barcelona",
    "habitantes": 1620343
  },
  {
    "ciudad": "París",
    "habitantes": 2103000
  }
]
```

5) Los nombres de las capitales que empiecen por la letra 'B' con el formato: nombre(país)

```
paises[$match(capital,/^B/i)].(capital & " (" & nombre & ")")
```

```
[
  "Brasilia (Brasil)",
  "Beijing (China)"
]
```

6) Los datos de los países ordenados descendentemente por población con el formato *pais(población)*

(OJO) tal como se ha hecho la función de ordenación, ordena por población de más a menos, por lo que no hay que hacer posteriormente el *$reverse*.

```
$sort(paises, function($l, $r) {
    $l.numero_de_habitantes < $r.numero_de_habitantes    }).( nombre & "(" & numero_de_habitantes & ")")
```

```
[
  "China(1400050000)",
  "India(1380004385)",
  "Estados Unidos(331002651)",
  "Brasil(212559417)",
  "Francia(67081000)",
  "España(46754783)",
  "Australia(25687041)"
]
```

7) El número de personas que hablan los distintos idiomas en cada país con el formato:

```
{
  "pais": "x" ,
  "idiomas": [   {
                   "idioma": "y",
                   "num_hablantes": z
                 }
             ]
}
```

(OJO) Este ejemplo tiene un concepto que es importante que tengamos claro. Cuando estamos procesando el array de *"idiomas",* dentro de ese array no tengo acceso al dato del "numero_de_habitantes" del país que estoy procesando, ya que ese dato está en el 'padre'. Esa es la razón, por la que se ha creado una función auxiliar a la que le pasamos el array de los idiomas y el número de habitantes.

```
(
  $procesarIdiomas := function($datos, $numHabitantesPais) {
    $datos.{
      "idioma": idioma,
      "num_hablantes": $round($numHabitantesPais * (porcentaje / 100))
    }
  };

  paises.{
    "pais": nombre,
    "idiomas": $procesarIdiomas(idiomas, numero_de_habitantes)
  }
)
```


```
[
  {
    "pais": "España",
    "idiomas": [
      {
        "idioma": "Español",
        "num_hablantes": 46287235
      },
      {
        "idioma": "Catalán",
        "num_hablantes": 7948313
      },
      {
        "idioma": "Gallego",
        "num_hablantes": 2337739
      },
      {
        "idioma": "Euskera",
        "num_hablantes": 1402643
      }
    ]
  },
  {
    "pais": "Estados Unidos",
    "idiomas": [
      {
        "idioma": "Inglés",
        "num_hablantes": 264802121
      },
      {
        "idioma": "Español",
        "num_hablantes": 43030345
      }
    ]
```


8) Obtener, utilizando la función **$reduce**, los nombres de los países en los que al menos dos idiomas sean hablados por más del 40% de su población.

```
paises[$reduce(idiomas, function($acum,$dato){$dato.porcentaje>40 ? $acum + 1 : $acum  } , 0 )>=2].nombre
```

```
"India"
```

9) Hacer lo mismo que en la consulta 9, pero utilizando las funciones **$count** y **$filter**, en lugar de **$reduce**.

```
paises[$count($filter(idiomas, function($dato) { $dato.porcentaje > 40 })) >= 2].nombre
```

```
"India"
```

10) Contar el número de idiomas de cada país con el formato {“pais”:”X”, “numIdiomas”:X} ordenados alfabéticamente.

```
$sort(paises.{"pais":nombre, "numIdiomas":$count(idiomas)},function($l, $r) { $l.pais > $r.pais} )
```


```
[
  {
    "pais": "Australia",
    "numIdiomas": 3
  },
  {
    "pais": "Brasil",
    "numIdiomas": 2
  },
  {
    "pais": "China",
    "numIdiomas": 3
  },
  {
    "pais": "España",
    "numIdiomas": 4
  },
  {
    "pais": "Estados Unidos",
    "numIdiomas": 2
  },
  {
    "pais": "Francia",
    "numIdiomas": 1
  },
  {
    "pais": "India",
    "numIdiomas": 3
  }
]
```


8.2 Transformación de documentos

JSONata no es propiamente, una herramienta que esté pensada para transformar un formato de documento como JSON a otros como XML, HTML, CSV, …. Está pensada para transformar documentos JSON y adaptar la salida a alguno de los tipos de datos con los que trabaja JSON, ya sabemos: *arrays*, *objetos*, *String*, etc. Por eso, **la transformación de JSON a JSON resulta natural** y se consiguen rápidamente los resultados esperados y, sin embargo, **la transformación de JSON a otros formatos hay que hacerla de forma indirecta a través del String**, ya que podemos incluir en su interior el contenido total del fichero transformado, o también, podríamos hacerlo como **un *array de Strings*, si lo quiero representar línea a línea**.

1) **(JSON → JSON)** Comenzaremos con un ejemplo sencillo, en el que construiremos los datos de los países con un formato distinto al original. Queremos que el nuevo formato sea:

```
{
 "infoPaises": [
  {
    "país": "X",
    "continente": "X",
    "habitantes": X,
    "capital": "X",
    "ciudades": [ "X1", "X2",  … ],
    "idiomaMasHablado": "X"
  },
  …
  …
 ]
}
```

(OJO), Para saber cuál es el idioma más hablado, se puede ordenar de mayor a menor y quedarnos con el primero.

```
{"infoPaises": paises.{
                "pais":nombre,
                "continente":continente,
                "habitantes":numero_de_habitantes,
                "capital":capital,
                "ciudades":principales_ciudades.nombre,
                "idiomaMasHablado":$sort(idiomas, function($l, $r){ ($r.porcentaje > $l.porcentaje) and $r.idioma!="Otros" })[0].idioma
              }
}
```

```
{
  "infoPaises": [
    {
      "pais": "España",
      "continente": "Europa",
      "habitantes": 46754783,
      "capital": "Madrid",
      "ciudades": [
        "Madrid",
        "Barcelona",
        "Valencia",
        "Sevilla"
      ],
      "idiomaMasHablado": "Español"
    },
    {
      "pais": "Estados Unidos",
      "continente": "América del Norte",
      "habitantes": 331002651,
      "capital": "Washington D.C.",
      "ciudades": [
        "Nueva York",
        "Los Ángeles",
        "Chicago",
        "Houston"
      ],
      "idiomaMasHablado": "Inglés"
    },
    {
```

2) **(JSON → JSON)** En este caso queremos obtener un nuevo documento JSON que requerirá procesamiento extra. Debe mostrar los datos de los diferentes idiomas que aparecen en el JSON original y un array con datos de los países en los que se habla ese idioma. El formato debe ser el siguiente:

```
[{
  "idioma": "XXX",
  "paises": [
   {
     "pais": "XXX",
     "poblacion_pais": XXX,
     "porcentaje_hablantes": XXX,
     "num_hablantes": XXX
   }
    ...
    ...
  ]
 },
 ...
 ...
]
```

Quizás, la mayor dificultad radica en que en este ejemplo, la raíz del documento cambia respecto al documento original. En el documento original, los datos estaban organizados por países, mientras que, en este caso, los datos estarán organizados por idiomas.

```
(
    /* Obtengo el array de los distintos idiomas que aparecen en el JSON ordenados alfabéticamente */
    $arIdiomas := $sort($distinct(paises.idiomas.idioma));

    /* Escribo una función que: */
    /* ---> Recibe por parámetro el nombre de un idioma */
    /* ---> Devuelve objetos con los datos de ese idioma en los distintos países en los que se habla */
    $tratarIdioma := function($idioma){
        paises[idiomas[idioma=$idioma]].{
                "pais":nombre,
                "poblacion_pais": numero_de_habitantes,
                "porcentaje_hablantes" : idiomas[idioma=$idioma].porcentaje ,
                "num_hablantes": $round(numero_de_habitantes * idiomas[idioma=$idioma].porcentaje /100)
        }
    };

    /* Voy recorriendo el array con los nombres de los distintos idiomas e invoco a la función que lo trata */
    $map($arIdiomas,function($valor){
        {
            "idioma" : $valor,
            "paises": [$tratarIdioma($valor)]
        }
    })
)
```

JSONa
2.0.6

El resultado obtenido tiene la siguiente forma:

```
[
  {
    "idioma": "Cantonés",
    "paises": [
      {
        "pais": "China",
        "poblacion_pais": 1400050000,
        "porcentaje_hablantes": 5,
        "num_hablantes": 70002500
      }
    ]
  },
  {
    "idioma": "Catalán",
    "paises": [
      {
        "pais": "España",
        "poblacion_pais": 46754783,
        "porcentaje_hablantes": 17,
        "num_hablantes": 7948313
      }
    ]
  },
```


3) **(JSON → CSV)** Vamos a ver ahora como se podría generar un fichero csv en el que almacenemos la siguiente información: *"Nombre"*, *"Continente"*, *"Capital"*, *"Población"* y *"Superficie"*. Para que el ejemplo quede más completo, vamos a definir una función en la que podamos indicar el separador a usas y si queremos cabecera o no. Le vamos a añadir también un parámetro adicional que nos indique si queremos el resultado como una única *String* o como un *array de Strings.*

```
(
  $generarFilas := function($datos,$sep){
    (
      $map($datos,function($pais){
        $pais.nombre & $sep & $pais.continente & $sep & $pais.capital & $sep & $pais.numero_de_habitantes & $sep & $pais.superficie
      } )
    )
  };

  $toCSV := function($datos, $sep, $quieroCabecera, $unaUnicaLinea){
    (
      /* Definimos cómo será la cabecera */
      $cabecera := "Nombre" & $sep & "Continente" & $sep & "Capital" & $sep & "Población" & $sep & "Superficie";

      /* Obtenemos un array con la información de los distintos países */
      $filas := $generarFilas($datos,$sep);

      /* Generamos la variable $salida en la le añadimos al resultado la cabecera si procede*/

      $salida := $quieroCabecera ? $append([$cabecera], $filas) : $filas;

      /* Como $salida tiene en este momento un array[ ] de String, concatenamos en un String sus diferentes elementos si queremos una única línea */
      $unaUnicaLinea ? $join($salida, "\n") : $salida
    )
  };

  /* Llamamos a la función indicando el separador que queremos usar, si queremos cabecera y si queremos una salida en un único String o en un array de String*/
  $toCSV(paises, ":", false, false)
)
```


```
[
  "España:Europa:Madrid:46754783:505990",
  "Estados Unidos:América del Norte:Washington D.C.:331002651:9833520",
  "Brasil:América del Sur:Brasilia:212559417:8515767",
  "Francia:Europa:París:67081000:551695",
  "India:Asia:Nueva Delhi:1380004385:3287263",
  "China:Asia:Beijing:1400050000:9596961",
  "Australia:Oceanía:Canberra:25687041:7692024"
]
```

4) **(JSON → HTML)** Vamos a generar en este ejemplo un String que represente una tabla HTML que contenga los siguientes campos: *"Nombre"*, *"Continente"*, *"Capital"*, *"Población"* y *"Superficie"*. Vamos a añadir a la función que crea el HTML un parámetro para indicar si queremos la salida como un array de Strings o como un único String.

```
(
  $generarFilas := function($datos,$sep){
      (
          $join($map($datos,function($pais){
            "<TR>" &
            "<TD>" & $pais.nombre & "</TD>" &
            "<TD>" & $pais.continente & "</TD>" &
            "<TD>" & $pais.capital & "</TD>" &
            "<TD>" & $pais.numero_de_habitantes & "</TD>" &
            "<TD>" & $pais.superficie & "</TD>" &
            "</TR>"
           } ),"")
      )
  };

  $toHTML := function($datos, $unaUnicaLinea){
      (

        $salida=[];
        /* Vamos a añadiendo al array que hay en la variable $salida, las distintas líneas que componen el HTML */
        $salida := $append($salida, "<HTML>");
        $salida := $append($salida, "<HEAD>");
        $salida := $append($salida, "<TITLE>Ejemplo de tabla</TITLE>");
        $salida := $append($salida, "</HEAD>");
        $salida := $append($salida, "<BODY>");
        $salida := $append($salida, '<TABLE border="1">');
        /* Insertamos la fila de cabecera de la tabla */
        $salida := $append($salida, "<TR><TH>Nombre</TH><TH>Continente</TH><TH>Capital</TH><TH>Población</TH><TH>Superficie</TH></TR>");
        /* Insertamos la información de las filas de los distintos países */
        $salida := $append($salida,$generarFilas($datos,$sep));
        $salida := $append($salida, "</TABLE>");
        $salida := $append($salida, "</BODY>");
        $salida := $append($salida, "</HTML>");

        /* Como $salida tiene en este momento un array[ ] de String, concatenamos en un String sus diferentes elementos si queremos una única línea */
        $unaUnicaLinea ? $join($salida, "") : $salida
      )
   };

  /* Llamamos a la función indicando el separador que queremos usar, si queremos cabecera y si queremos una salida en un único String o en un array de String*/
  $toHTML(paises, true)
)
```

JSONata 2.0.6

```
"<HTML><HEAD><TITLE>Ejemplo de tabla</TITLE></HEAD><BODY><TABLE border=\"1\"><TR><TH>Nombre</TH><TH>Continente</TH><TH>Capital</TH><TH>Población</TH><TH>Superficie</TH></TR><TR><TD
```

Si visualizamos la salida en un navegador (quitándole las comillas del inicio y el fin), vemos lo siguiente:

Nombre	Continente	Capital	Población	Superficie
España	Europa	Madrid	46754783	505990
Estados Unidos	América del Norte	Washington D.C.	331002651	9833520
Brasil	América del Sur	Brasilia	212559417	8515767
Francia	Europa	París	67081000	551695
India	Asia	Nueva Delhi	1380004385	3287263
China	Asia	Beijing	1400050000	9596961
Australia	Oceanía	Canberra	25687041	7692024